

Deep Learning Topics: Special Connections Part II Transformers

Paul Rodriguez, PhD

(SDSC)

June 2026



Outline

- **Basic word prediction task and motivating the attention strategy**
- **A basic Attention Head network and exercise**
- **Transformers**
- **BERT and GPT strategies**
- **Transformers in Science Applications**
- **Combining Images and Text**

Dependences of Language

Consider this sequence:

*The Law will never be perfect, but it's application
should be just - this is what we are missing, in my
opinion <End of Sequence>*

What does 'it' refer to that can have an 'application'?

Dependences of Language

Consider this sequence:

*The Law will never be perfect, but it's application
should be just - this is what we are missing, in my
opinion <End of Sequence>*

What does 'it' refer to that can have an 'application'?

e.g 'it' refers back to 'Law', which is part of 'the Law'
noun phrase, which is the entity that will 'never be
perfect', and so on ...

Dependences of Language

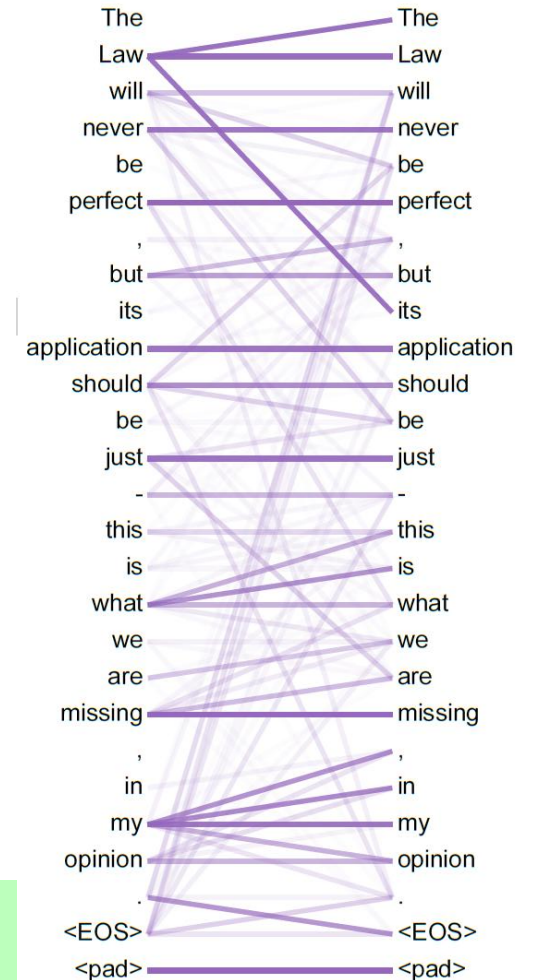
Consider this sequence:

The Law will never be perfect, but it's application should be just - this is what we are missing, in my opinion <End of Sequence>

What does 'it' refer to that can have an 'application'?

e.g 'it' refers back to 'Law', which is part of 'the Law' noun phrase, which is the entity that will 'never be perfect', and so on ...

In language, there are many dependencies and interactions between words



A toy problem to get some intuition

- Let's use the following list of 7 tokens (ie vocabulary of 7 words)
1 <start>, 2 the, 3 man, 4 chicken, 5 ordered, 6 woman, 7 beef
- For input data, let's have these 2 sentences and token id sequences:
 <start> man ordered the chicken [1,3,5,2,4]
 <start> woman ordered the beef [1,6,5,2,7]
- **Now let's try to predict the next word by 'attention' idea**

The toy task: predict next word

Predict the next word for the first input sentence after each word:

<Start> man ordered the chicken

The toy task: predict next word

Predict the next word for the first input sentence after each word:
<Start> man ordered the chicken

A basic solution is a bigram matrix:
X=Sequence-to-Word, size is $T \times V$

Token
Sequence

Next Token (ie word) Prediction

Pos	Word	<strt>	The	Man	Chik	Ord	Wmn	Beef
1	<start>			0.5			0.5	
2	Man					1.0		
3	Orde.r		1.0					
4	The				0.5			0.5
5	Chick.	1.0						

The toy task: predict next word

Predict the next word for the first input sentence after each word:
<Start> man ordered the chicken

A basic solution is a bigram matrix:
X=Sequence-to-Word, size is TxV

Token
Sequence

Next Token (ie word) Prediction

Pos	Word	<strt>	The	Man	Chik	Ord	Wmn	Beef
1	<start>			0.5			0.5	
2	Man					1.0		
3	Orde.r		1.0					
4	The				0.5			0.5
5	Chick.	1.0						

*The bigram doesn't
use context*

The toy task: predict next word

Predict the next word for the first input sentence after each word:
<Start> man ordered the chicken

A basic solution is a bigram matrix:
X=Sequence-to-Word, size is TxV

Challenge, can we learn predictions ($\rightarrow$) that depend on context of other tokens and/or position

After 'Man' the $\rightarrow$ chicken = 1.0

After 'Woman' the $\rightarrow$ beef = 1.0

Next Token (ie word) Prediction

Pos	Word	<strt>	The	Man	Chik	Ord	Wmn	Beef
1	<start>			0.5			0.5	
2	Man					1.0		
3	Orde.r		1.0					
4	The				0.5			0.5
5	Chick.	1.0						

nce

The toy task: predict next word

Predict the next word for the first input sentence after each word:
<Start> man ordered the chicken

A basic solution is a bigram matrix:
X=Sequence-to-Word, size is TxV

Challenge, can we learn predictions ($\rightarrow$) that depend on context of other tokens and/or position

After 'Man' the $\rightarrow$ chicken = 1.0

After 'Woman' the $\rightarrow$ beef = 1.0

Next Token (ie word) Prediction

Pos	Word	<strt>	The	Man	Chik	Ord	Wmn	Beef
1	<start>			0.5			0.5	
2	Man				1.0			
3	Orde.r		1.0					
4	The				0.5			0.5
5	Chick.	1.0						

We want 'man' context to pass information

The attention idea

Let's get all tokens to 'pass information' about dependencies

E.G. for X a $T \times V$ matrix of possible predictions, we want to transform X into *contextualized predictions*

$$\begin{array}{c} \mathbf{X = \text{Sequence-to-Word is } T \times V} \end{array} \quad \begin{array}{c} \text{Contextualized Predictions} \end{array}$$
$$\begin{pmatrix} 0 & 0 & 0.5 & 0 & 0 & 0.5 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0.5 & 0 & 0 & 0.5 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix} \rightarrow$$

The attention idea

Let's get all tokens to 'pass information' about dependencies

E.G. for X a $T \times V$ matrix of possible predictions, we want to transform X into *contextualized predictions*

We need a W a $T \times T$ matrix – aka 'attention' weights that has word dependencies

$$\begin{array}{c} \mathbf{X = \text{Sequence-to-Word is } T \times V} \end{array} \quad \begin{array}{c} \text{Contextualized Predictions} \end{array}$$
$$\begin{pmatrix} 0 & 0 & 0.5 & 0 & 0 & 0.5 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0.5 & 0 & 0 & 0.5 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix} \rightarrow$$

The attention idea

Let's get all tokens to 'pass information' about dependencies

E.G. for X a $T \times V$ matrix of possible predictions, we want to transform X into *contextualized predictions*

We need a W a $T \times T$ matrix – aka 'attention' weights that has word dependencies

$$\begin{array}{c} \text{X= Sequence-to-Word is } T \times V \end{array} \quad \begin{array}{c} \text{Contextualized Predictions} \end{array}$$
$$W_{T \times T} * \begin{pmatrix} 0 & 0 & 0.5 & 0 & 0 & 0.5 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0.5 & 0 & 0 & 0.5 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix} \Rightarrow \begin{pmatrix} 0 & 0 & 0.5 & 0 & 0 & 0.5 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

The attention idea

Let's get all tokens to 'pass information' about dependencies

E.G. for X a $T \times V$ matrix of possible predictions, we want to transform X into *contextualized predictions*

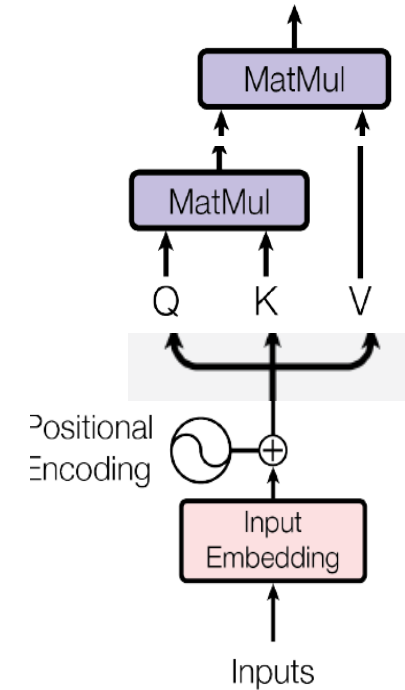
We need a W a $T \times T$ matrix – aka 'attention' weights that has word dependencies

$$\begin{array}{c} \mathbf{X} = \text{Sequence-to-Word is } T \times V \end{array} \quad \begin{array}{c} \text{Contextualized Predictions} \end{array}$$
$$W_{T \times T} * \begin{pmatrix} 0 & 0 & 0.5 & 0 & 0 & 0.5 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0.5 & 0 & 0 & 0.5 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix} \Rightarrow \begin{pmatrix} 0 & 0 & 0.5 & 0 & 0 & 0.5 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

W should reflect the interdependencies of the words in the input

Q: Where should W come from? (Hint: what does all AI need)

- Let's build up the attention architecture



Get input embeddings

First, get sequence of token embeddings using 1-hot input to E units – see next slide

$$[1,3,5,2,4] \rightarrow X_{T \times E}$$

Sidenote: embedding (encoding) for words (ie tokens)

Each word (or word part) is assigned a 'token id' 1 to V=50K

Let X be input vector with only one value equal to 1 (a 1-hot vector) determined by token id



X is Vx1

0	(
0	(
.	(
.	(
.	(
	(
	(
	(
0	(
1	(
0	(
.	(
.	(
.	(
	(
	(
0	(
0	(

Sidenote: embedding (encoding) for words (ie tokens)

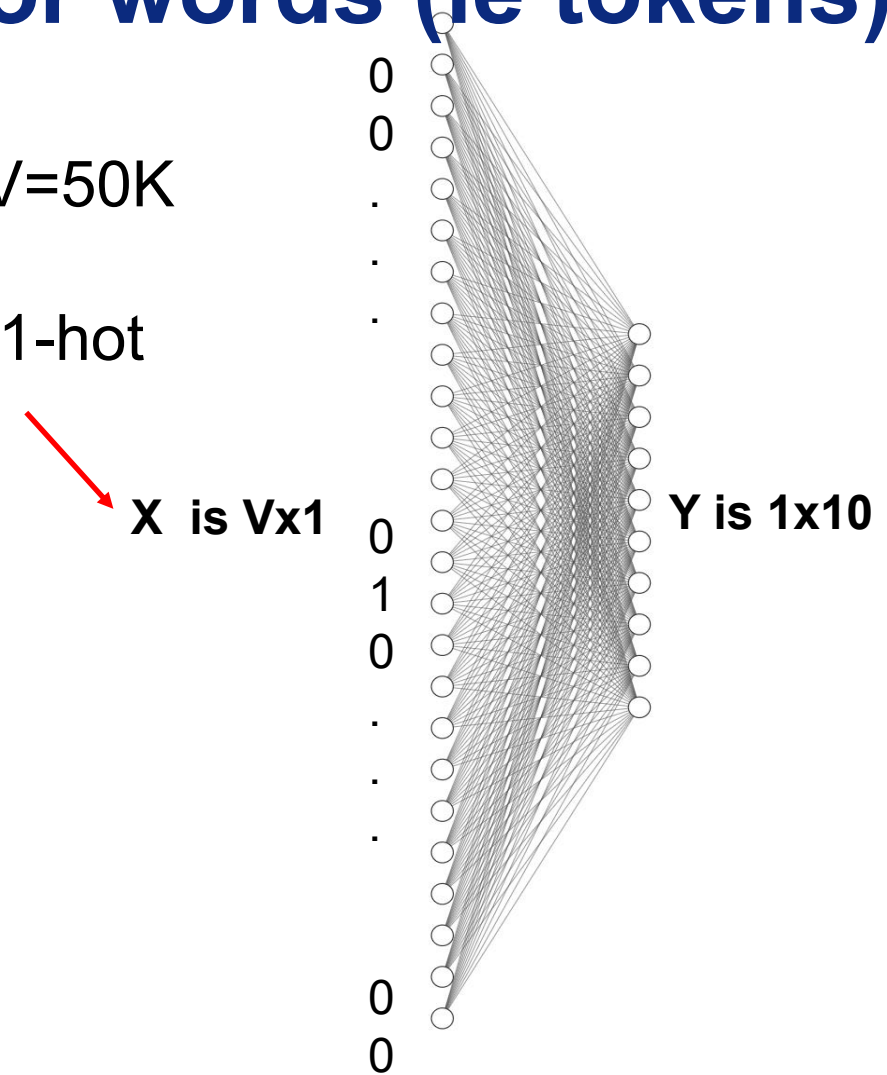
Each word (or word part) is assigned a 'token id' 1 to $V=50K$

Let X be input vector with only one value equal to 1 (a 1-hot vector) determined by token id

Use a one layer network to make an embedding

In this picture, each token id is converted to a lower dimensional vector with size 1×10

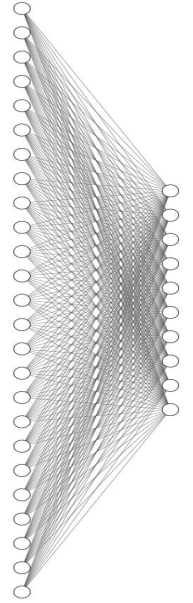
Each token sequence of T vectors is $T \times 10$



(back to) Get input embeddings

First, get sequence of token embeddings using 1-hot input to E units

$$[1,3,5,2,4] \rightarrow X_{T \times E}$$



Get input and add in position info

First, get sequence of token embeddings using 1-hot input to E units

$$[1,3,5,2,4] \rightarrow X_{T \times E}$$

Then do the same for positions 1...T

$$[1,2,3,4,5] \rightarrow P_{T \times E}$$

Get input and add in position info

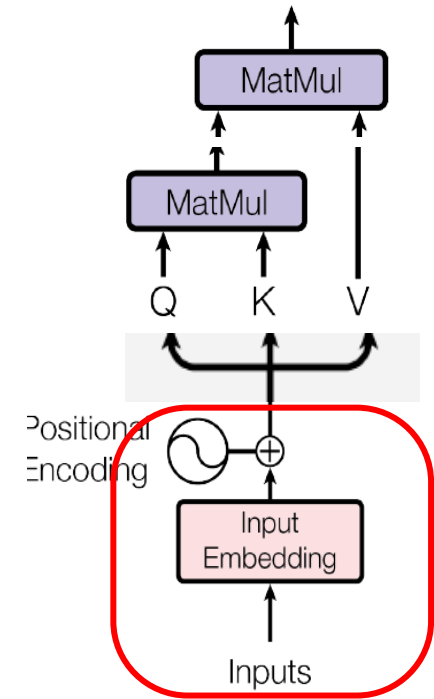
First, get sequence of token embeddings using 1-hot input to E units

$$[1,3,5,2,4] \rightarrow X_{T \times E}$$

Then do the same for positions 1...T

$$[1,2,3,4,5] \rightarrow P_{T \times E}$$

$X + P$ is final $T \times E$ matrix of input embeddings



Embeddings for attention weights

Take Input Embeddings and build a 'Query' (Q) and 'Key' (K) embedding matrix of size $T \times H$

$$\begin{aligned} X + P &\rightarrow Q_{T \times H} \\ X + P &\rightarrow K_{T \times H} \end{aligned}$$

The idea is to make new embeddings that can be useful for word-to-word dependencies

Embeddings for attention weights

Take Input Embeddings and build a 'Query' (Q) and 'Key' (K) embedding matrix of size $T \times H$

$$X + P \rightarrow Q_{T \times H}$$
$$X + P \rightarrow K_{T \times H}$$

The idea is to make new embeddings that can be useful for word-to-word dependencies

Now let $W = Q * K'$

Notice that every token's embedding gets to 'interact' with every other token's embedding to make up the $T \times T$ elements of W

Embeddings for attention weights

Take Input Embeddings and build a 'Query' (Q) and 'Key' (K) embedding matrix of size $T \times H$

$$\begin{aligned} X + P &\rightarrow Q_{T \times H} \\ X + P &\rightarrow K_{T \times H} \end{aligned}$$

The idea is to make new embeddings that can be useful for word-to-word dependencies

Now let $W = Q * K'$

Notice that every token's embedding gets to 'interact' with every other token's embedding to make up the $T \times T$ elements of W

$$X + P \rightarrow V_{T \times V}$$

Finally, instead of a pre-built bigram matrix, use another embedding for a 'Value' V matrix

Embeddings for attention weights

Take Input Embeddings and build a 'Query' (Q) and 'Key' (K) embedding matrix of size $T \times H$

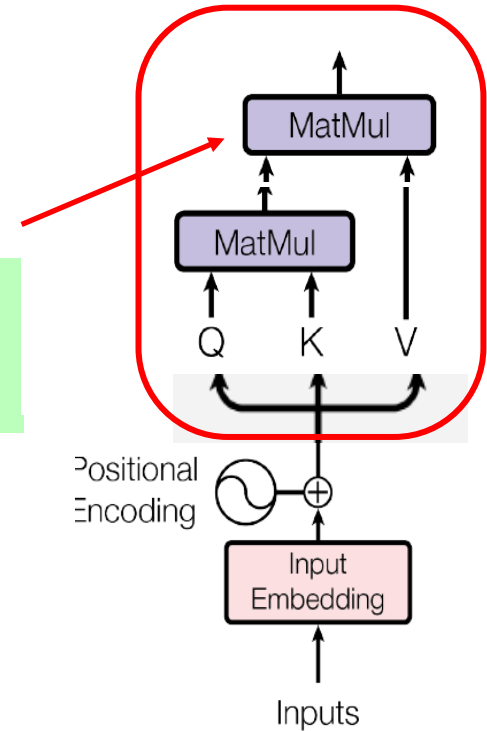
$$X + P \rightarrow Q_{T \times H}$$

$$X + P \rightarrow K_{T \times H}$$

Now, we can take sigmoid of $W * V$ to get $T \times V$ predictions from 0 to 1

Now let $W = Q * K'$

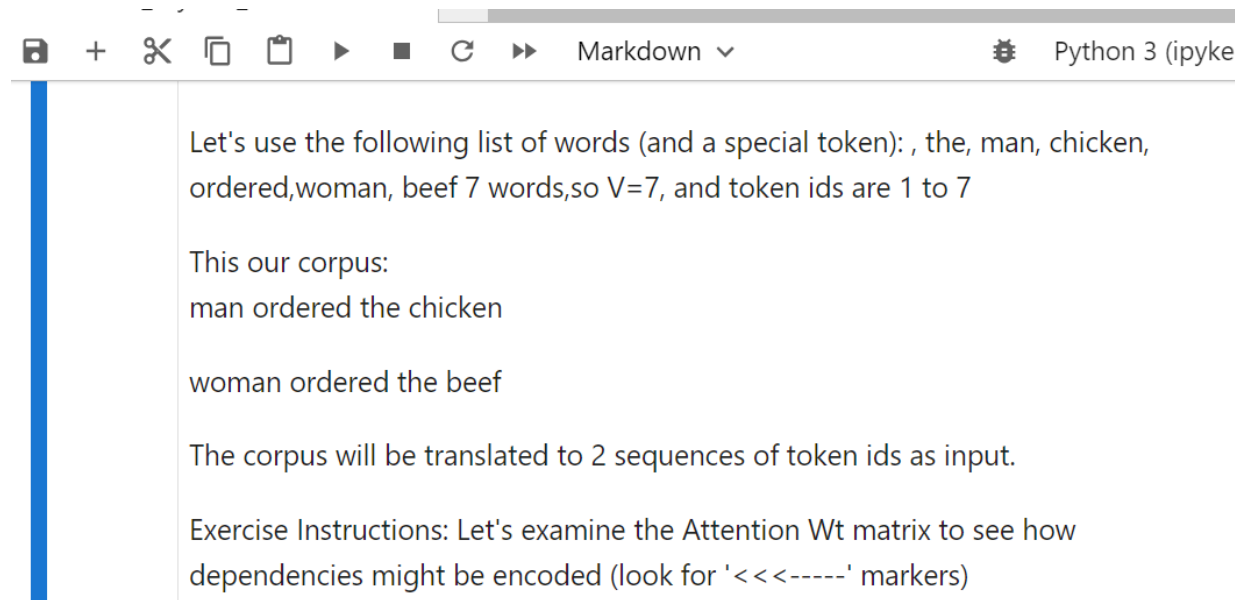
$$X + P \rightarrow V_{T \times V}$$



In class exercise:

An example of attention head with a toy task:

Run the “toytask_attention notebook” and observe the printed predictions and attention weights.



```
Let's use the following list of words (and a special token): , the, man, chicken,
ordered,woman, beef 7 words,so V=7, and token ids are 1 to 7

This our corpus:
man ordered the chicken

woman ordered the beef

The corpus will be translated to 2 sequences of token ids as input.

Exercise Instructions: Let's examine the Attention Wt matrix to see how
dependencies might be encoded (look for '<<-----' markers)
```

Output TxV predictions

Notice <start> → [man or woman at 0.50]

Notice that the → [chicken or beef] predictions change depending on who's ordering

	<ST>	the	man	chkn	ordrd	woman	beef
0 <ST>	0.00	0.00	0.49	0.00	0.00	0.49	0.00
1 man	0.00	0.01	0.00	0.00	0.98	0.00	0.00
2 ordrd	0.00	0.99	0.00	0.00	0.00	0.00	0.00
3 the	0.02	0.00	0.00	0.96	0.00	0.00	0.02
4 chkn	0.98	0.00	0.00	0.01	0.00	0.00	0.00

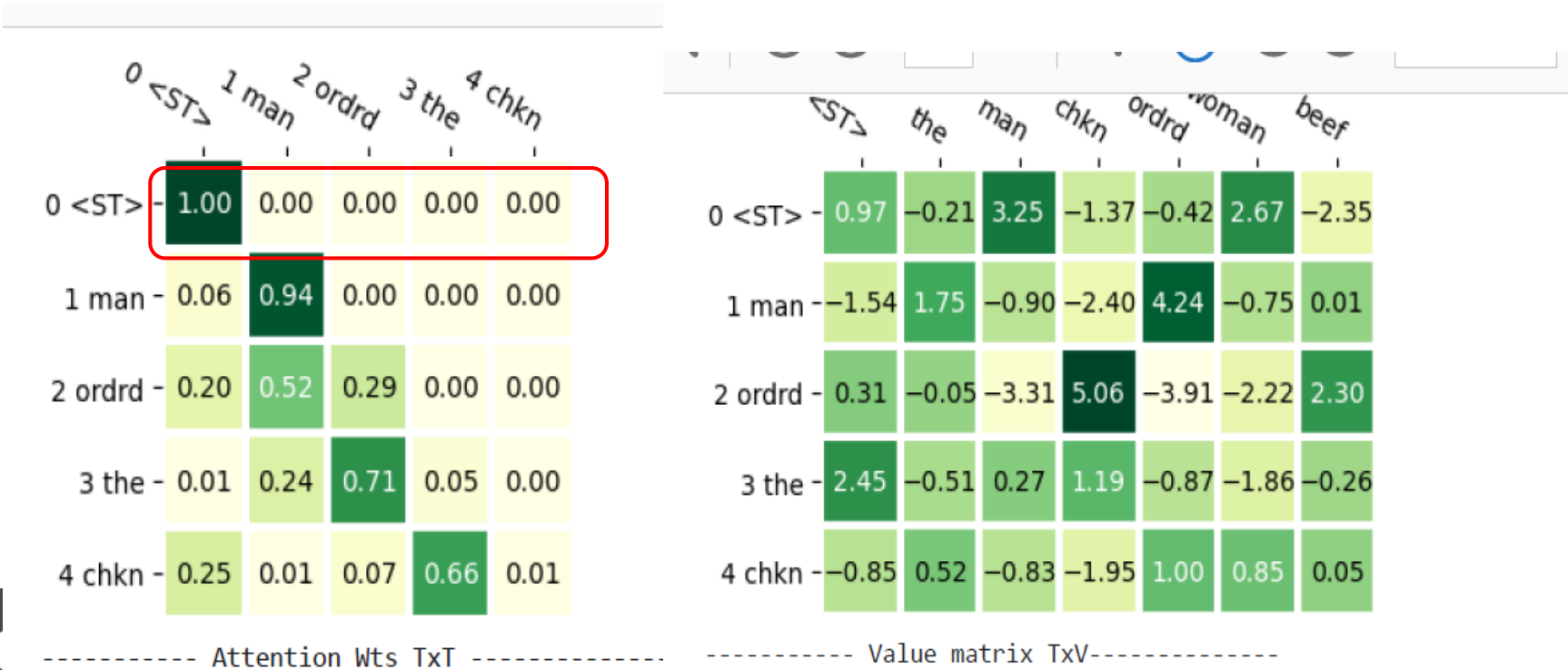
----- Output Predictions TxV (t-th row are prediction

	<ST>	the	man	chkn	ordrd	woman	beef
0 <ST>	0.00	0.00	0.49	0.00	0.00	0.49	0.00
1 woman	0.00	0.01	0.00	0.00	0.97	0.00	0.02
2 ordrd	0.00	0.99	0.00	0.00	0.00	0.00	0.00
3 the	0.00	0.01	0.00	0.02	0.03	0.00	0.95
4 beef	0.98	0.00	0.00	0.01	0.00	0.00	0.00

Here are the W (attention) and V (value) matrices for [<st> man ordrd the chkn]

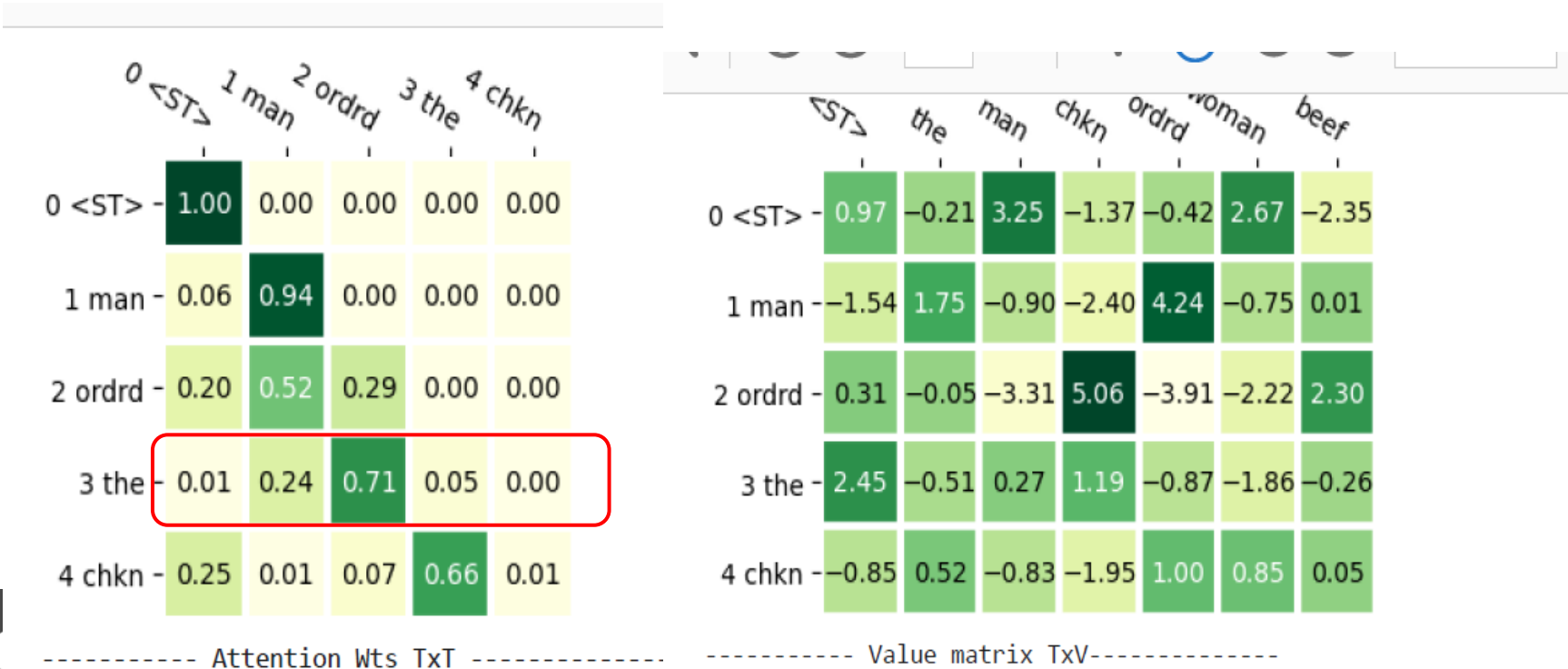
Notice the upper diagonal of W are masked so that predictions don't “look ahead”

Notice the <Start> token only depends on itself, and only picks off top row of V



Here are the W (attention) and V (value) matrices for [<st> man ordrd the chkn]

The 4th row of W 1xT times V makes the 1xV predictions of ‘the’



Here are the W (attention) and V (value) matrices for [<st> man ordrd the chkn]

The 4th row of W 1xT times V makes the 1xV predictions of ‘the’

only 2 values
in 4th row > 0

	0 <ST>	1 man	2 ordrd	3 the	4 chkn
0 <ST>	1.00	0.00	0.00	0.00	0.00
1 man	0.06	0.94	0.00	0.00	0.00
2 ordrd	0.20	0.52	0.29	0.00	0.00
3 the	0.01	0.24	0.71	0.05	0.00
4 chkn	0.25	0.01	0.07	0.66	0.01

----- Attention Wts TxT -----

	<ST>	the	man	chkn	ordrd	woman	beef
0 <ST>	0.97	-0.21	3.25	-1.37	-0.42	2.67	-2.35
1 man	-1.54	1.75	-0.90	-2.40	4.24	-0.75	0.01
2 ordrd	0.31	-0.05	-3.31	5.06	-3.91	-2.22	2.30
3 the	2.45	-0.51	0.27	1.19	-0.87	-1.86	-0.26
4 chkn	-0.85	0.52	-0.83	-1.95	1.00	0.85	0.05

----- Value matrix TxV -----

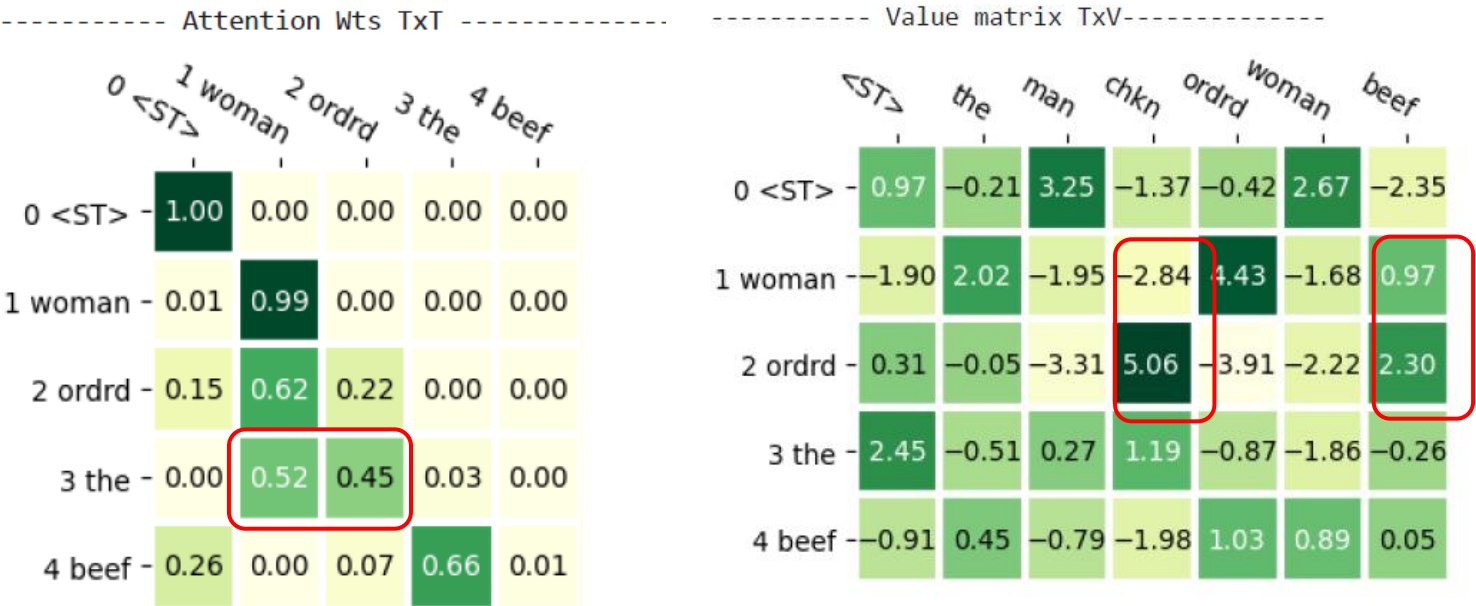
let's focus on
chkn, beef
predictions

Here are the W (attention) and V (value) matrices for [**<st> wmn ordrd the beef**]:

The 4th row of W 1xT times V makes the 1xV predictions of ‘the’

And let’s focus on ‘chkn’ vs ‘beef’

only 2 values
in 4th row > 0



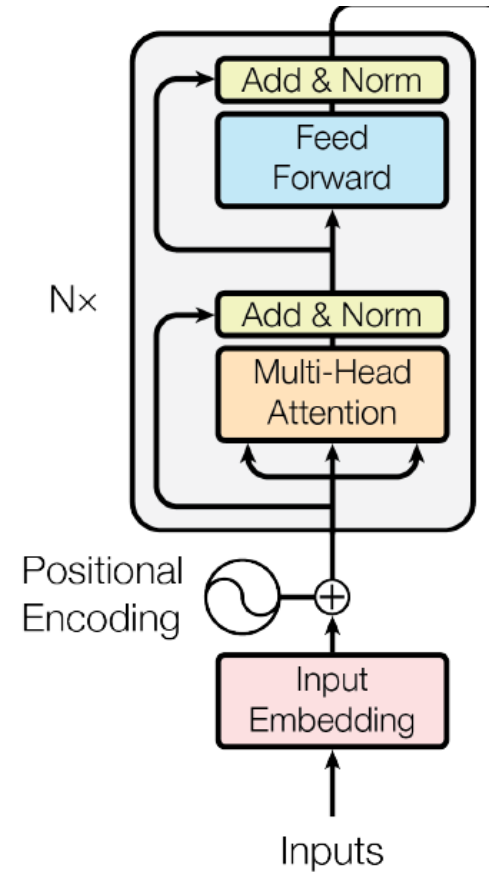
Pause –

- **Consider what if the man & woman orders switched for different restaurants?**
- **Let scale up to transformers**

Finally, a transformer

Include skip-add connections

Include Layer Normalization or DropOut layers

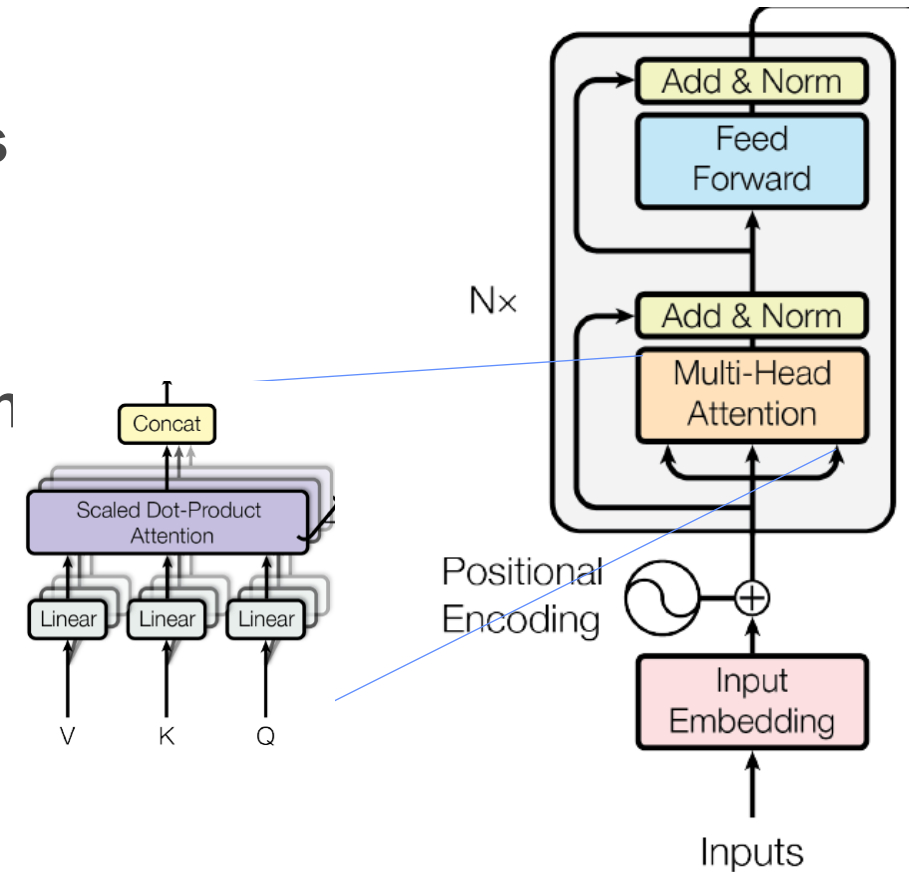


Finally, a transformer

Include skip-add connections

Include Layer Normalization or DropOut layers

Multi-Head – for N heads produce $T \times (H/N)$ each



Finally, a transformer

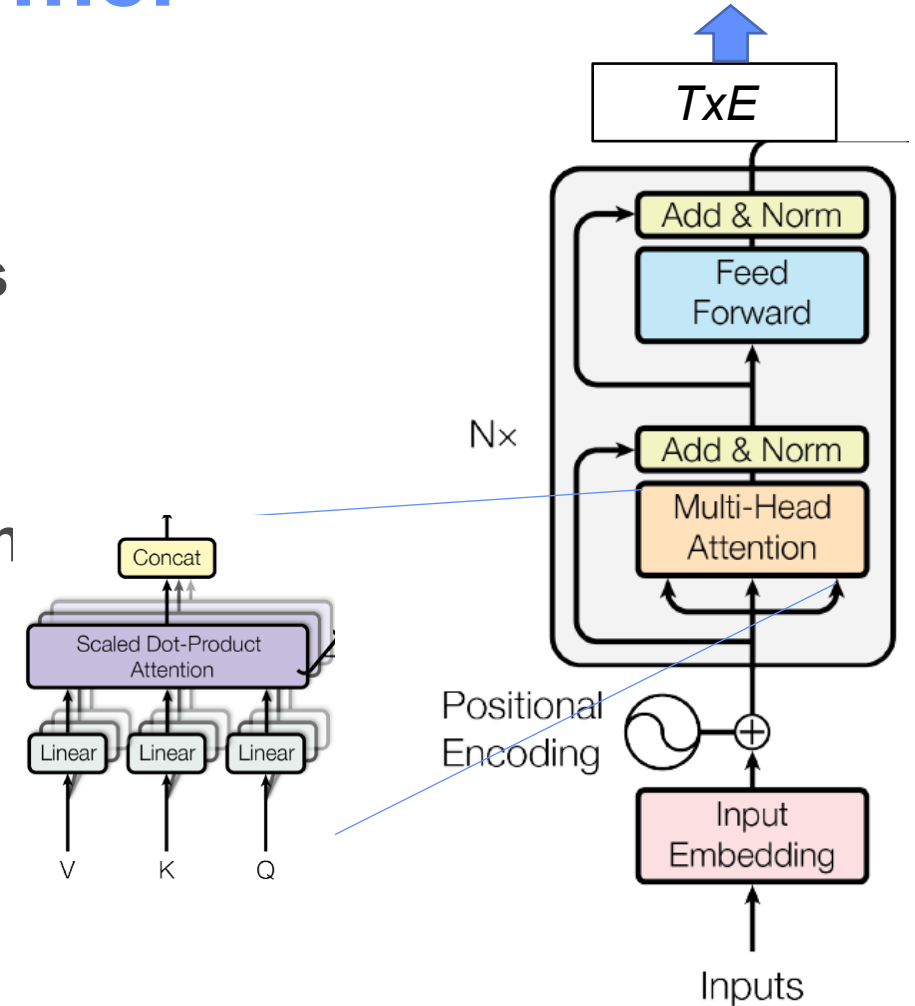
Include skip-add connections

Include Layer Normalization or DropOut layers

Multi-Head – for N heads produce $T \times (H/N)$ each

Add MLP layers on top –
output another $T \times E$ matrix

stackable!



Finally, a transformer

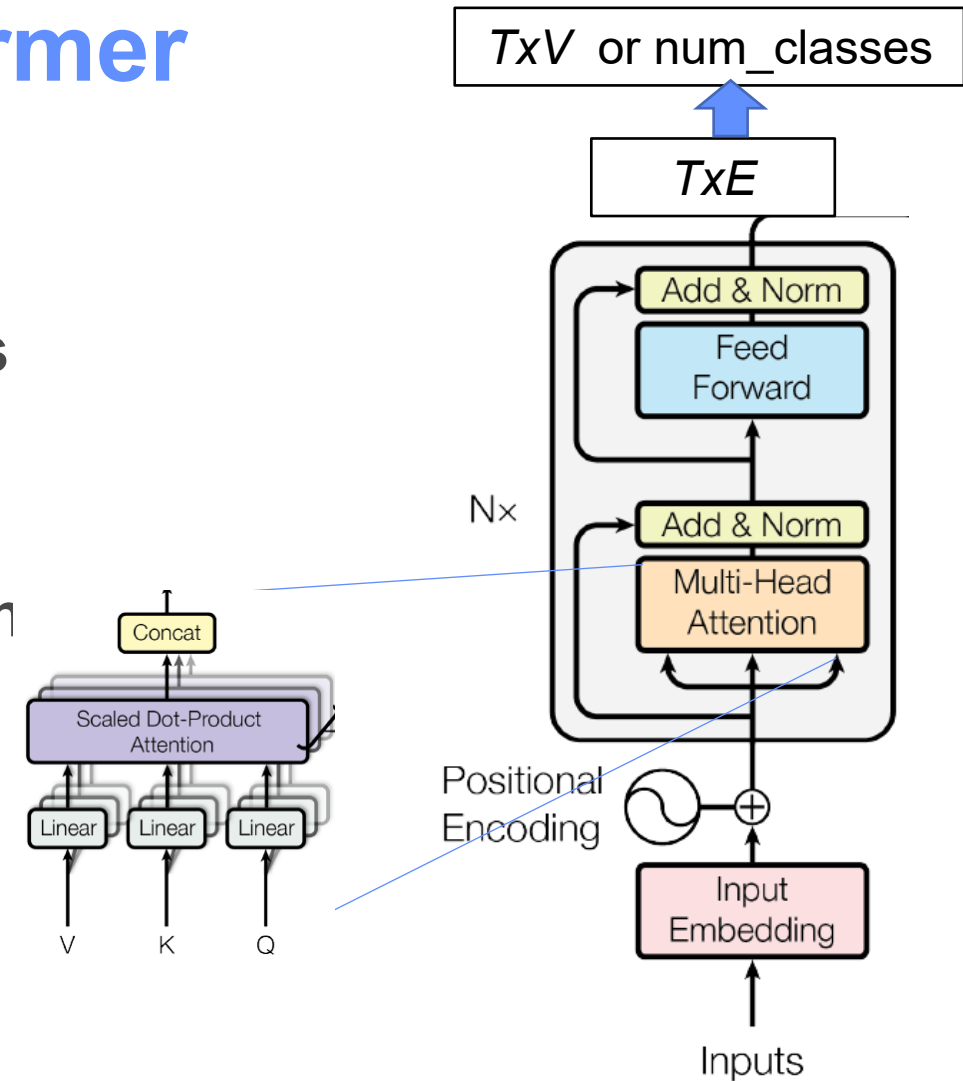
Include skip-add connections

Include Layer Normalization or DropOut layers

Multi-Head – for N heads produce $T \times (H/N)$ each

Add MLP layers on top –
output another $T \times E$ matrix
or output final probabilities

stackable!



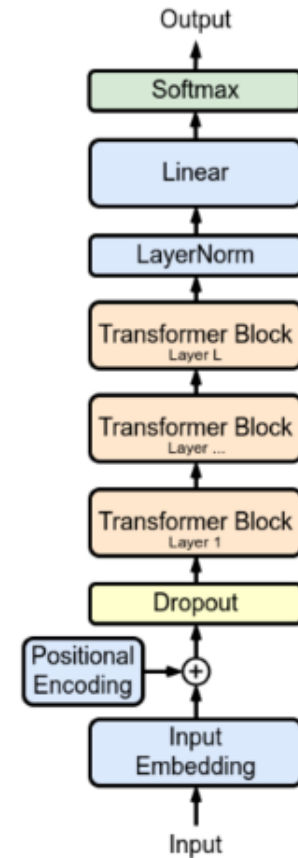
2 kinds of training strategies

GPT – predict next word only look back at prior context (which could be a whole document)

Put mask on attention weights so that predictions only depend on previous tokens

BERT – *No attention mask* so all token dependencies can influence all other tokens predictions

Special tokens help create a variety of tasks

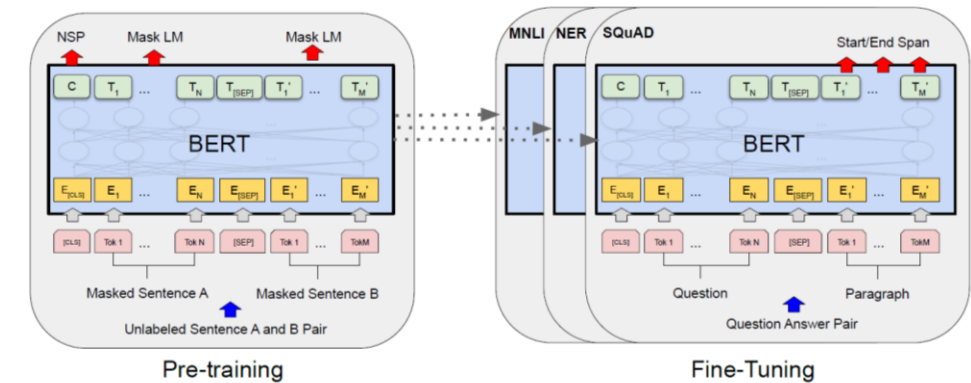


BERT (Bidirectional Encoder Representations from Transformers)

Goal: Train a model to develop general token-level AND sentence-level encoding

1 Pretrain on:

- fill-in-the-blank
- binary classification if 2 sentences go together



Devlin, etal, 2019

2 Fine tune on variety of tasks

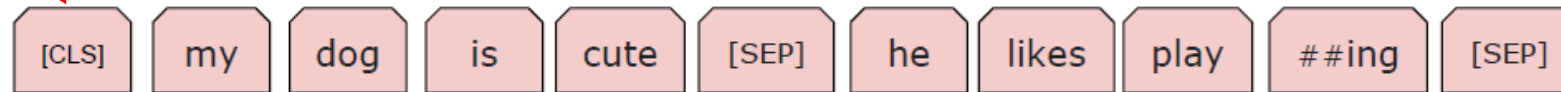
BERT Input: 2 sentences

[CLS]
is 'dummy' token
and always
present

[SEP] marks separation
between sentences or
sequences

[PAD] tokens help
fill out sequence to T
items

Input



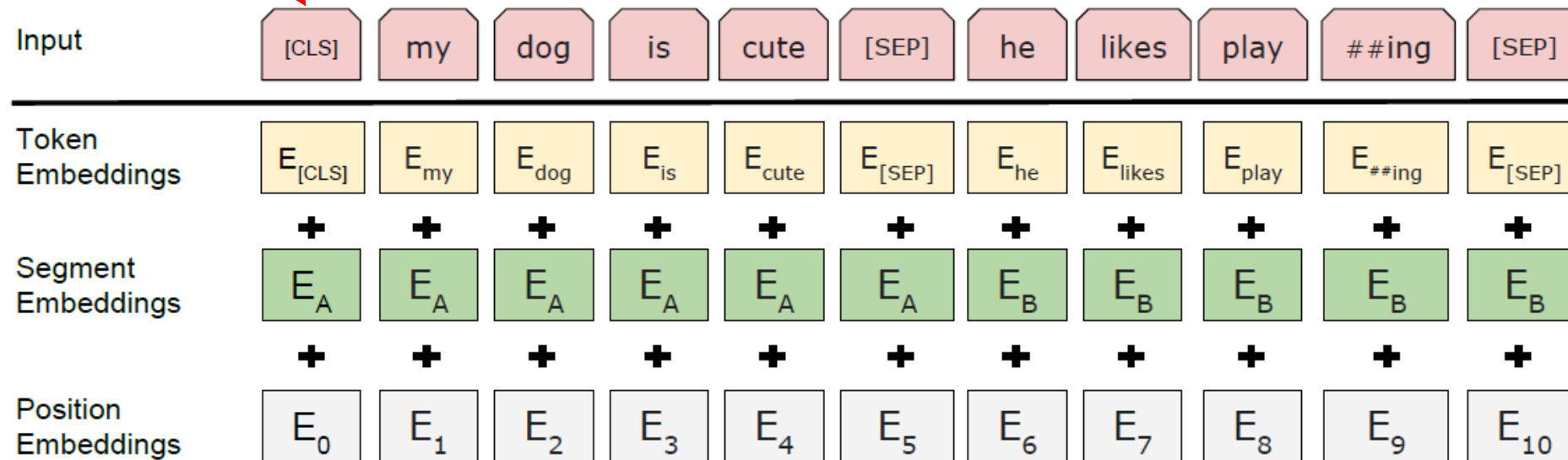
Devlin, etal, 2019

BERT Input: 2 sentences

[CLS]
is 'dummy' token
and always
present

[SEP] marks separation
between sentences or
sequences

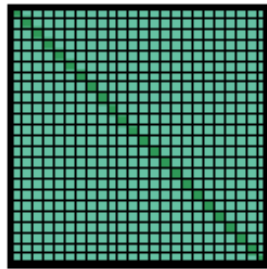
[PAD] tokens help
fill out sequence to T
items



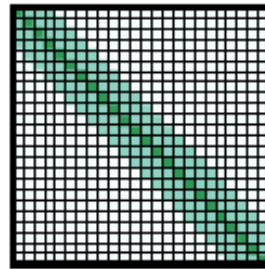
Add a TxE
matrix for 'A'
vs 'B' sentence

Other strategies: Sparse Attention

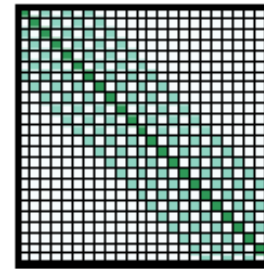
Mistral - long sequence sparse, large attention matrix for 'Longformer' or 'Sparse Transformer'



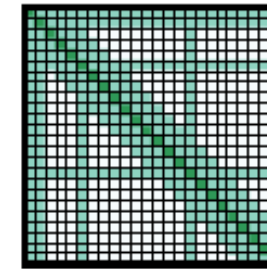
(a) Full n^2 attention



(b) Sliding window attention



(c) Dilated sliding window



(d) Global+sliding window

Figure 2: Comparing the full self-attention pattern and the configuration of attention patterns in our Longformer.

Generating Long Sequences with Sparse Transformers, Child et al, 2019
Longformer: The Long-Document Transformer Beltagy et al. Allen Institute, 2020

- **Transformers for Science applications**

Can anything be cast as a kind of sentence, or an arrangement of tokens?

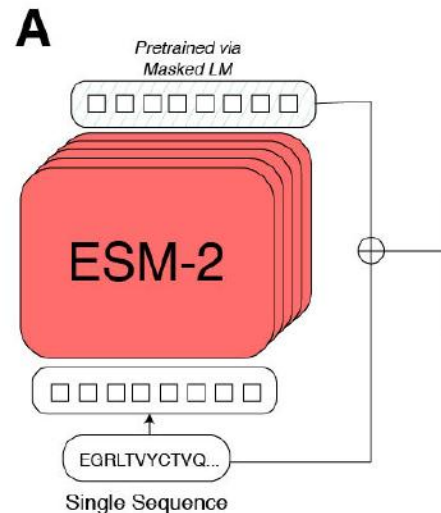
ESM Fold model

Language models of protein sequences at the scale of evolution enable accurate structure prediction

Lin et al, Meta Research 2022

Atom level structure prediction

Uses protein sequence as input to transformer layers (like LLM)



ESM Fold model

Language models of protein sequences at the scale of evolution enable accurate structure prediction

Lin et al, Meta Research 2022

- Atom level structure prediction
- Uses protein sequence as input to transformer layers (like LLM) then **feed Attention Matrix** to 'folding block'
- Folding block and a structure module iteratively refines a predicted map of protein contact (similar to AlphaFold2, but faster)

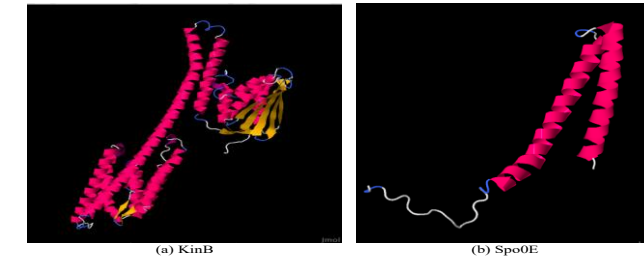
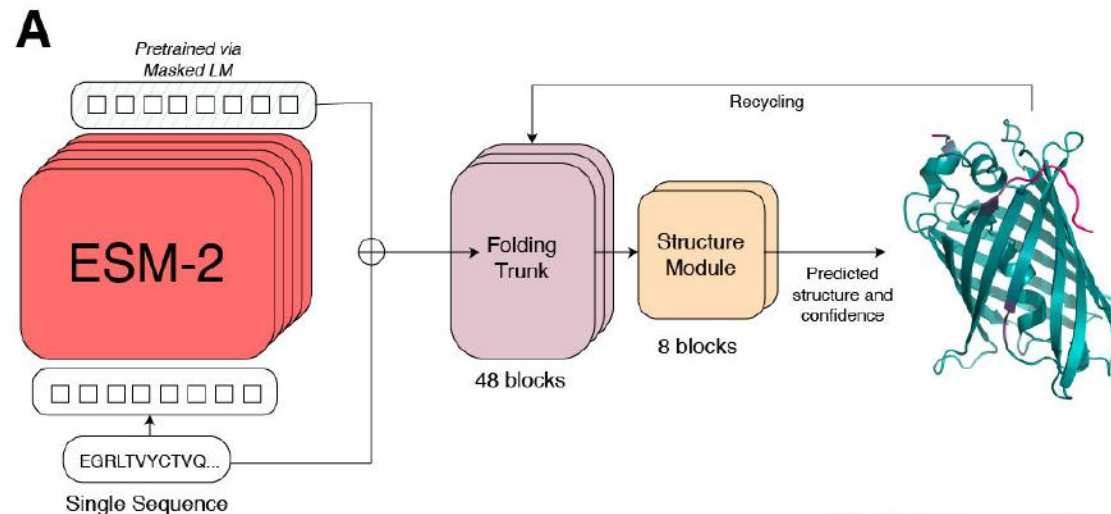


Figure 1. Protein Structures.

The two protein structures presented here KinB and Spo0E were generated by providing sequence of ~300 amino acids to ESMFold. (M.Gujral SDSC)

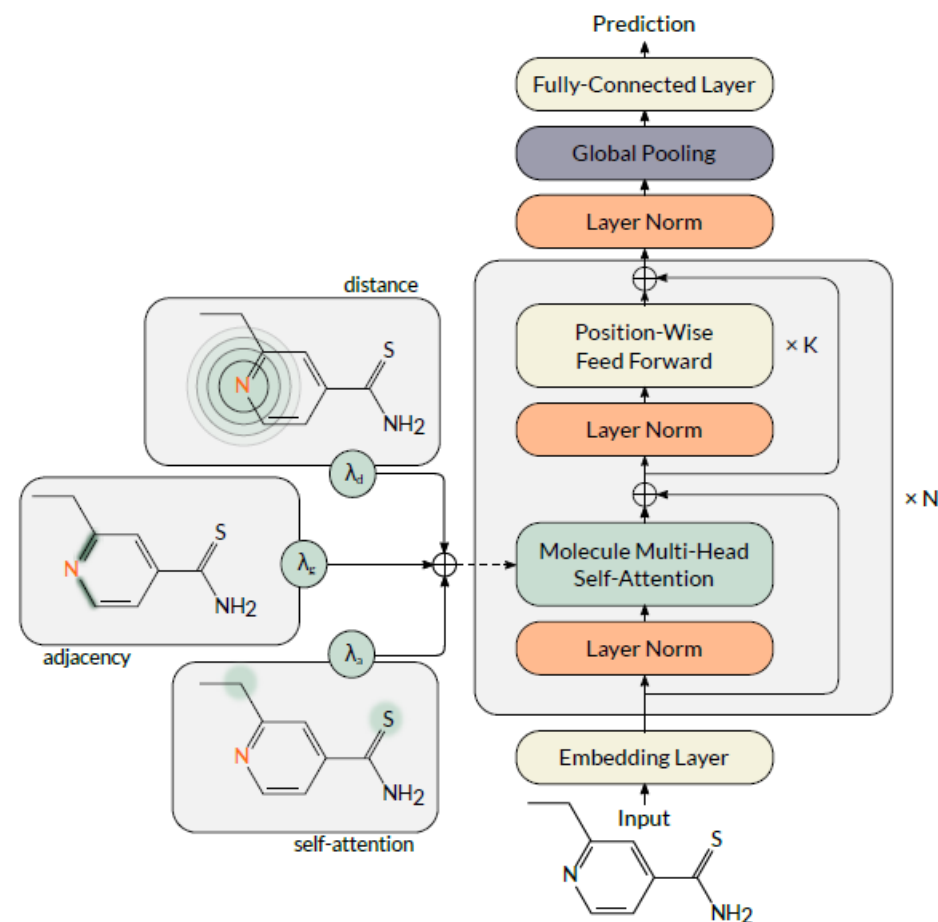
Molecule Attention Transformer

(Maziarka et al. 2020)

Molecular property prediction

Uses the set of atoms as input (like sentence tokens)

Includes spatial information by using a sum of the attention matrix, a distance matrix, and an adjacency matrix.



The Visual Transformer (ViT)

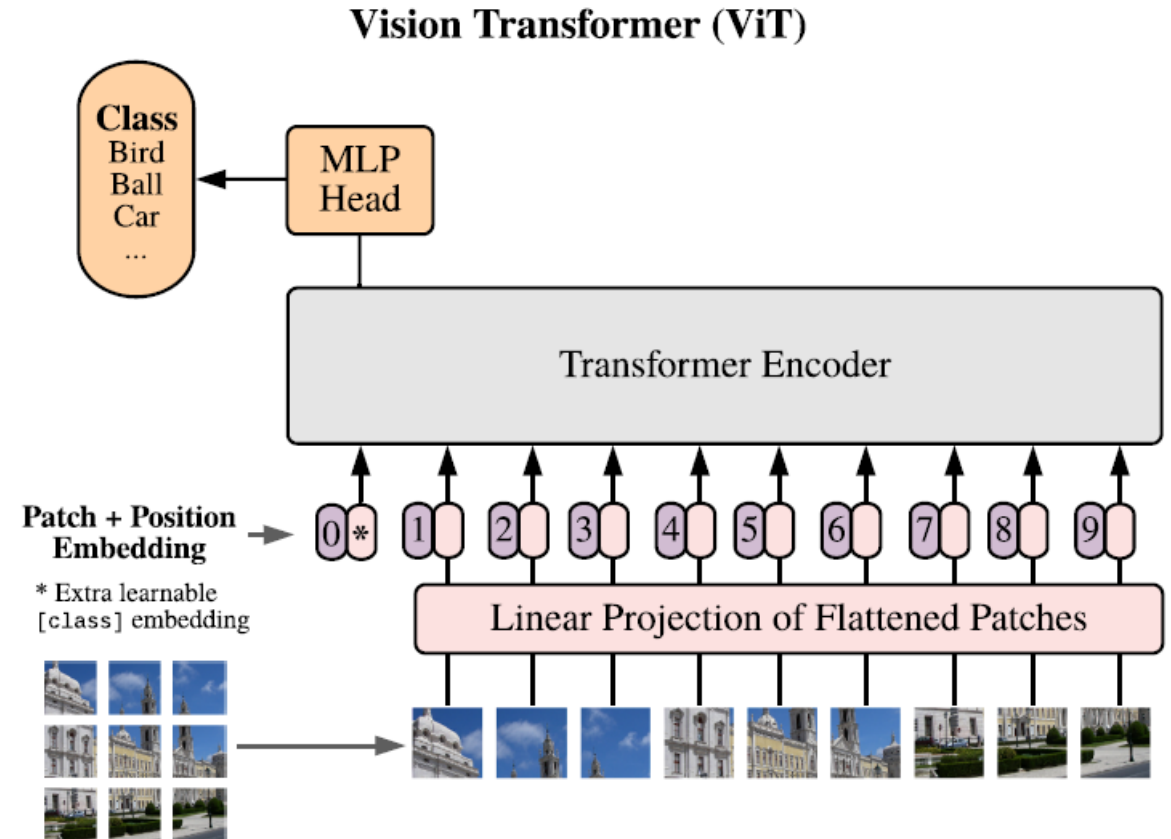
An image is worth 16x16 words: Transformers for image recognition at scale

Adosovitski, et al, 2021, Google Research

Uses a sequences of image patches (16x16) like a sentence of tokens (ie 224x224 pixels is 16x16 patches of 14x14 pixels)

Uses a classification token like Bert to learn image output classes

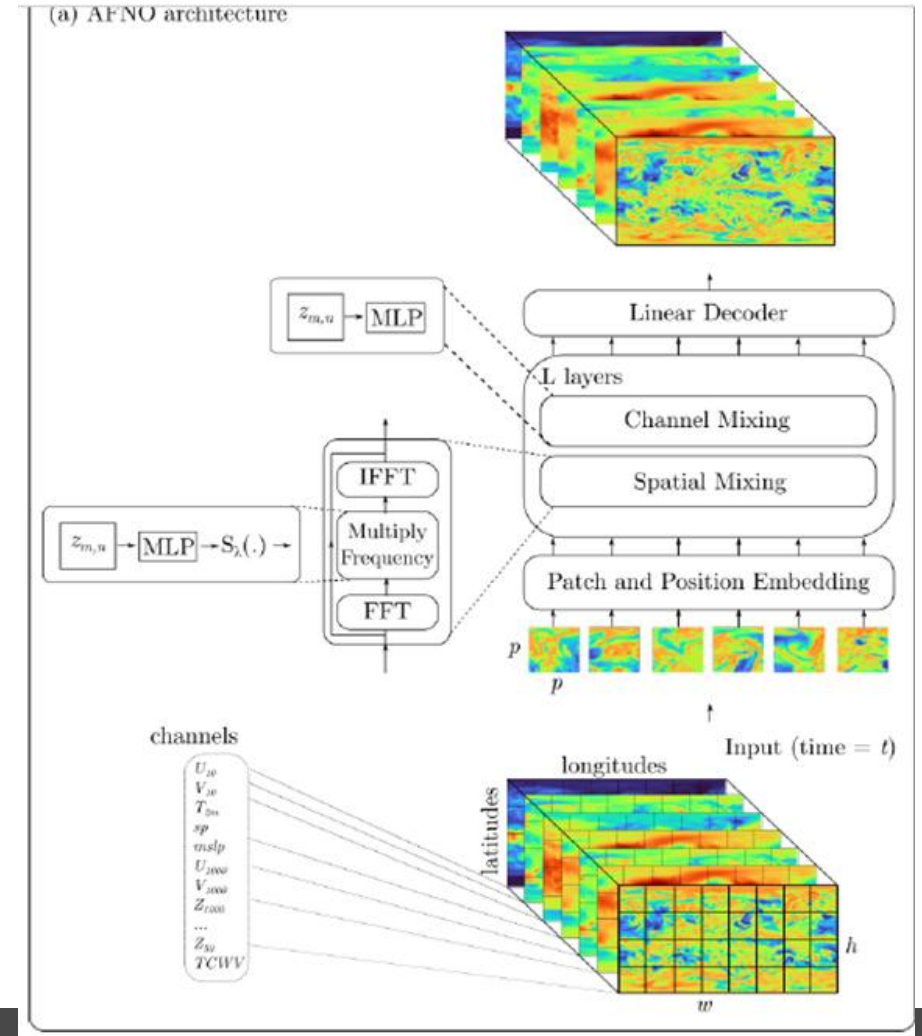
Competitive or better than CNNs but might need more data



Adaptive Fourier Neural Operator for weather prediction

Luca Delle Monache, Pat Mulrooney, Agniv Sengupta, SIO, UCSD

- Like a ViT starts with image patch and position embedding of atmospheric images
- Then uses FFT for ‘embedding/encoding’
- Instead of ‘attention’ matrix is uses an MLP to model interdependencies (mixing) between FFT components
- (It’s not exactly a transformer but a similar strategy. Also FFT for physical processes makes sense.



- **Combining images and text often makes DL work better, or more generic, for image or text tasks**

CLIP – Contrastive Language-Image Pretraining

Learning Transferable Visual Models From Natural Language Supervision

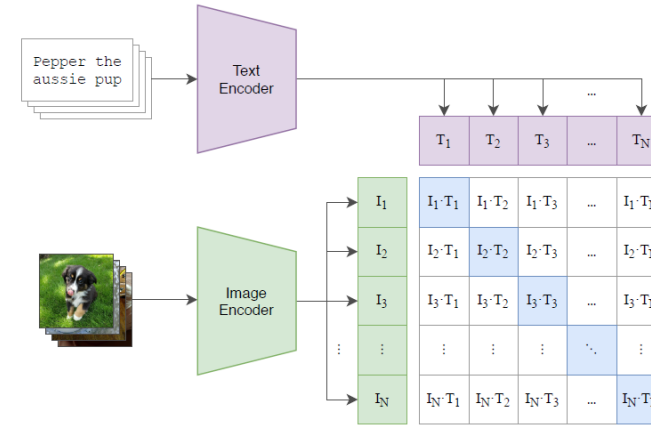
Radford et al, 2021, Open AI

Uses 400M images and captions for training

Learns a multi-modal embedding

Maximizes embedding similarity of captions with it's image; minimizes embedding similarity of captions with other images

(1) Contrastive pre-training



CLIP – Contrastive Language-Image Pretraining

Learning Transferable Visual Models From Natural Language Supervision

Radford et al, 2021, Open AI

Uses 400M images and captions for training

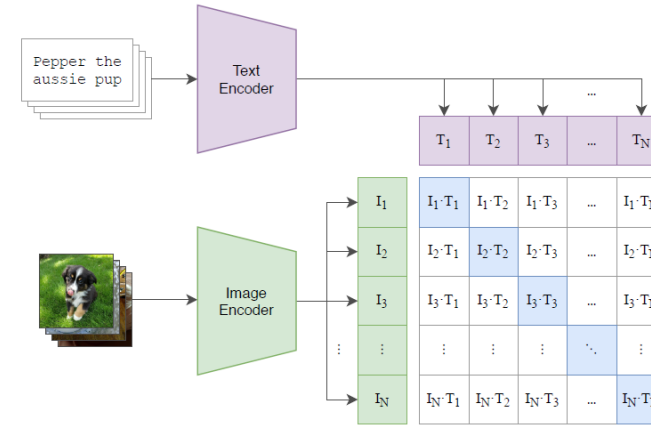
Learns a multi-modal embedding

Maximizes embedding similarity of captions with it's image; minimizes embedding similarity of captions with other images

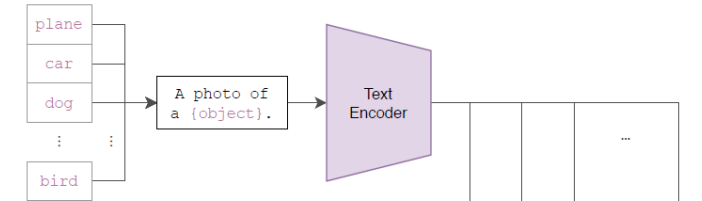
Performs classification by prompting it with an image and possible captions

Note: CLIP with diffusions gets close to DALL-E

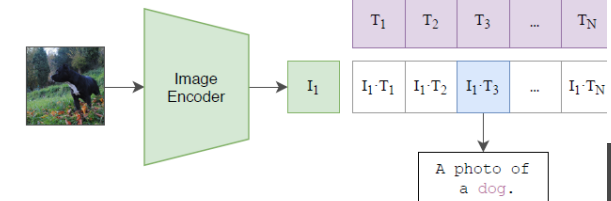
(1) Contrastive pre-training



(2) Create dataset classifier from label text



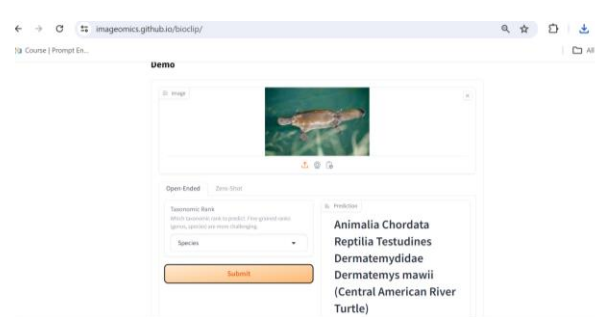
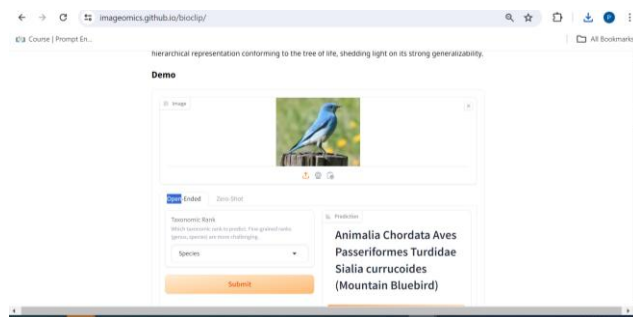
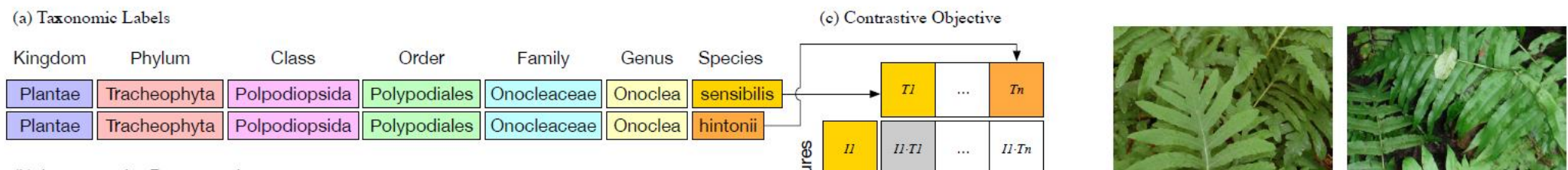
(3) Use for zero-shot prediction



BIOCLIP: A Vision Foundation Model for the Tree of Life

Stevens, etal 2024 OSU

- Uses pre-trained CLIP for a base
- Uses Tree-of-Life 10M dataset of biology images with taxonomic labels
- The taxonomic hierarchy is presented as a sequence of words for different species



BIOCLIP: A Vision Foundation Model for the Tree of Life

The model has github repo site and can be installed on Expanse directly as per instructions (mostly):

(see bioclip commands text file)

1. Get into interactive session use jupyter-gpu-shared-pytorch
2. start singularity shell
3. pip install from github to a user folder
4. Export PYTHONPATH
5. Point program to your images and run

```
#1 Request a GPU node
#use jupyter alias for gpushared.. but don't get into notebook

#2 ssh into that node
squeue -u $USER
ssh exp-X-X

#Note: X-X should be the expanse node id numbers

#3 Load modules

module load gpu
module load slurm
module load singularitypro/3.11

#4 Run singularity shell command

singularity shell --nv /cm/shared/apps/containers/singularity/bv
```

```
Singularity> pip install git+https://github.com/Imageomics/pybioclip
```

```
Singularity> export PYTHONPATH=/home/$USER/Local_BioClip/local/lib/p
Singularity> echo $PYTHONPATH
/home/p4rodrig/Local_BioClip/local/lib/python3.10/dist-packages/
```

```
Singularity> python3 run_bc.py
open_clip_pytorch_model.bin: 100%|
open_clip_config.json: 100%|
txt_emb_species.npy: 100%|
txt_emb_species.json: 100%|
Sialia currucoides - 0.9975905418395996
Tersina viridis - 0.0009066067868843675
Eumyias thalassinus - 0.00020906853023916483
Coracias garrulus - 0.00014162158186081797
Gymnorhinus cyanocephalus - 0.00013305182801559567
Singularity>
```

end